Orquestração de Containers com Kubernetes

Infraestruturas de Computação na Nuvem, Aula 05

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

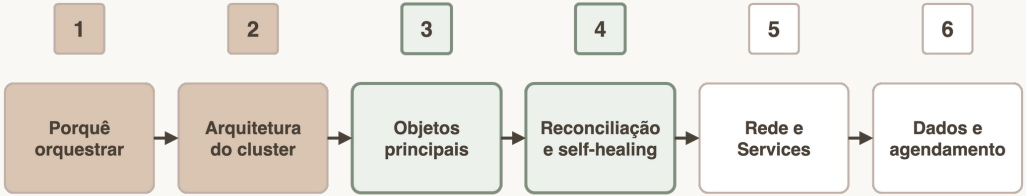
Agenda (1/2)

- Porque precisamos de orquestração além de um único host.
- Arquitetura Kubernetes: control plane e worker nodes.
- Objetos principais: Pod, Deployment, ReplicaSet, Service, Namespace.

Agenda (2/2)

- Ciclo de vida de um Pod e self-healing.
- O ciclo de reconciliação e o modelo declarativo.
- Tipos de Service e o modelo de rede do Kubernetes.
- Configuração, segredos e armazenamento persistente.
- Cargas de trabalho não contínuas: Job e CronJob.
- Agendamento: requests, limits e afinidade.
- Alternativas de orquestração: Swarm e Nomad.

Percurso da Aula

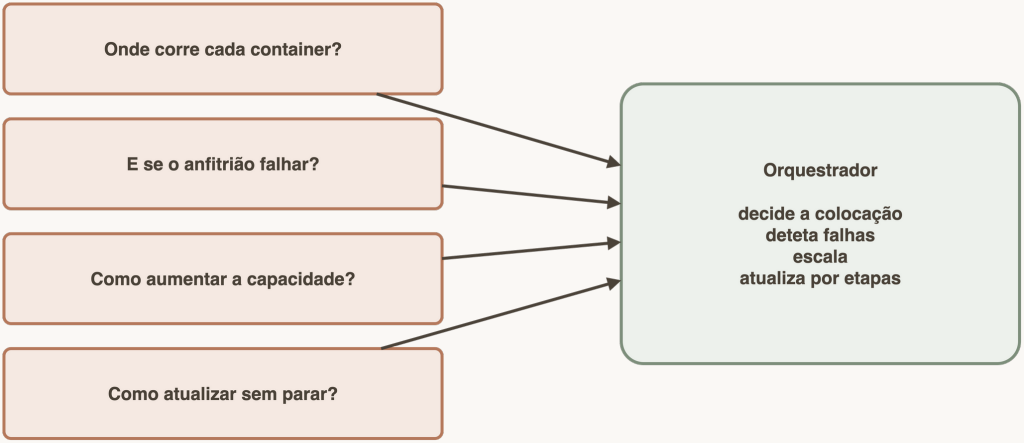


Do problema que a orquestração resolve ao funcionamento completo do cluster

Porque Precisamos de Orquestração

- O Docker, por si só, gere containers num único host — não decide onde correr cada container, nem reage a falhas.
- Aplicações em produção precisam de vários hosts, para tolerância a falhas e para distribuir carga.
- É necessário decidir automaticamente em que nó correr cada carga de trabalho (agendamento/scheduling).
- É necessário detetar falhas e recuperar delas sem intervenção manual.

As Questões que a Orquestração Resolve



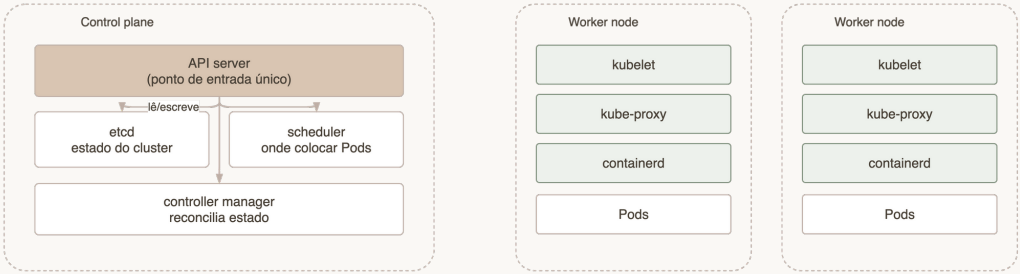
Arquitetura Kubernetes: Control Plane (1/2)

- O control plane toma as decisões globais sobre o cluster (agendamento, deteção e resposta a eventos).
- **API server**: ponto de entrada único; todas as interações com o cluster passam por aqui.
- **etcd**: base de dados chave-valor distribuída que guarda o estado completo do cluster.
- **scheduler**: decide em que worker node cada novo Pod deve ser colocado.

Arquitetura Kubernetes: Control Plane (2/2)

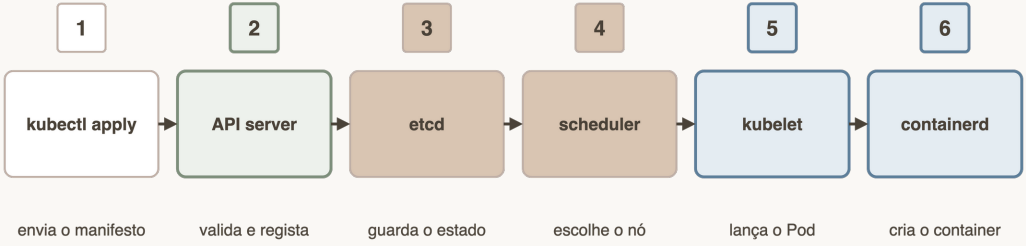
- **controller manager**: corre os controladores que comparam o estado atual com o estado desejado e agem para os aproximar.
- Os **worker nodes** são onde as cargas de trabalho efetivamente correm.
- **kubelet**: agente que garante que os containers descritos para o nó estão de facto a correr.
- **kube-proxy** e o **container runtime** (ex.: containerd) completam o nó, tratando de rede e execução de containers.

Arquitetura do Cluster



Toda a comunicação passa pelo API server. Os kubelets reportam o estado observado.

Do Pedido ao Container



O caminho de um pedido, do manifesto ao container em execução

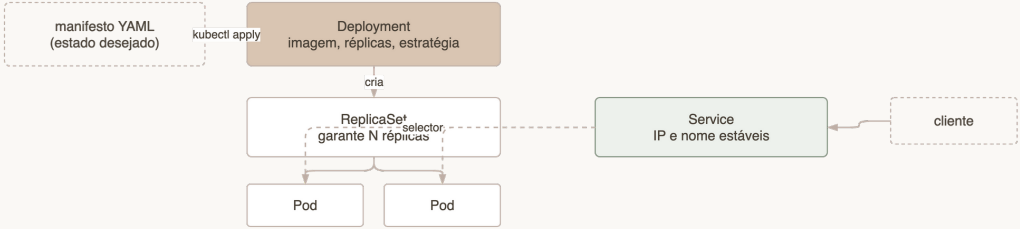
Objetos Principais do Kubernetes (1/2)

- **Pod**: unidade mínima de execução; agrupa um ou mais containers que partilham rede e armazenamento.
- **Deployment**: descreve o estado desejado de uma aplicação (imagem, número de réplicas, estratégia de atualização).
- O Deployment não gere Pods diretamente — delega isso num ReplicaSet.

Objetos Principais do Kubernetes (2/2)

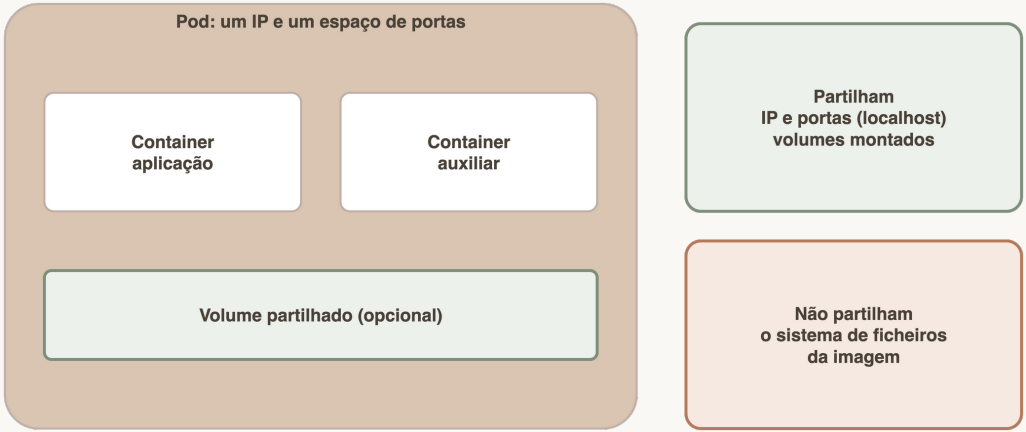
- **ReplicaSet**: garante que um determinado número de réplicas de um Pod está sempre em execução.
- **Service**: fornece um ponto de acesso estável (IP e nome) a um conjunto de Pods, que são efémeros por natureza.
- **Namespace**: divisão lógica do cluster, útil para isolar ambientes ou equipas dentro do mesmo cluster físico.

Do Manifesto ao Pod em Execução

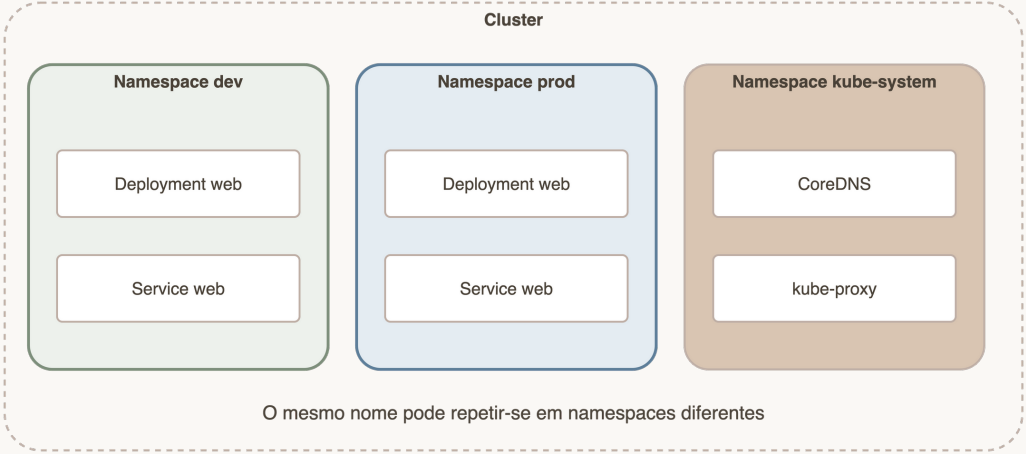


Apagar um Pod não o elimina: o ReplicaSet nota que falta uma réplica e cria outra.

Por Dentro de um Pod



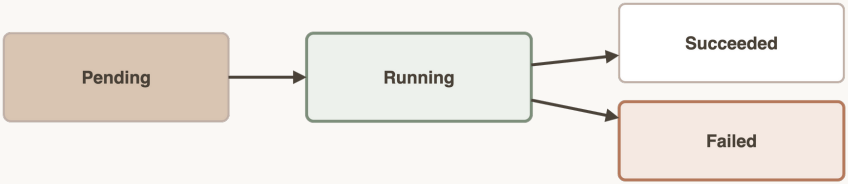
Namespaces do Kubernetes



Ciclo de Vida de um Pod e Self-Healing

- Um Pod passa por várias fases: Pending, Running, Succeeded, Failed.
- **Liveness probe**: verifica se o container continua saudável; se falhar, o kubelet reinicia-o.
- **Readiness probe**: verifica se o container está pronto para receber tráfego, antes de ser incluído num Service.
- Se um Pod morrer, o ReplicaSet cria automaticamente um novo, aproximando o cluster do estado desejado.

Fases do Pod e Sondas



Liveness
o container está vivo?
Se falha: o kubelet reinicia-o

Readiness
está pronto para tráfego?
Se falha: sai do Service

Declarativo vs Imperativo

- Modo imperativo: comandos diretos que dizem ao Kubernetes exatamente *o que fazer* (ex.: `kubectl run`, `kubectl create`).
- Modo declarativo: descreve-se o estado *desejado* num manifesto YAML e o Kubernetes encarrega-se de o atingir.
- `kubectl apply -f manifesto.yaml` aplica (ou atualiza) o estado descrito no ficheiro.
- Abordagem declarativa é a recomendada em produção: reprodutível, versionável e auditável.

Duas Formas de Pedir

Imperativo

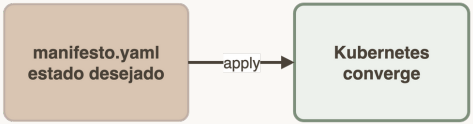
```
kubectl run ...
```

```
kubectl scale ...
```

```
kubectl set image ...
```

Sequência de ordens: depende da história

Declarativo

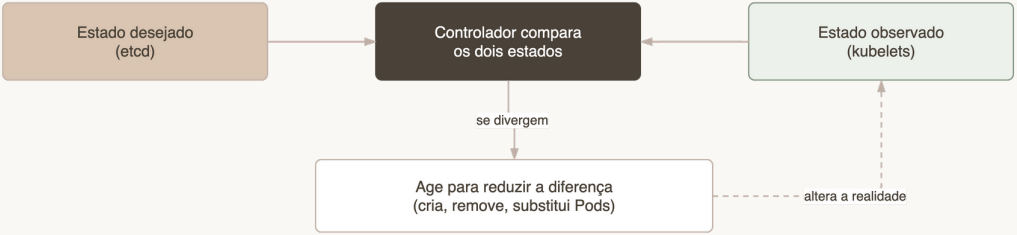


Descrição do resultado: reprodutível, versionável, auditável

O Ciclo de Reconciliação

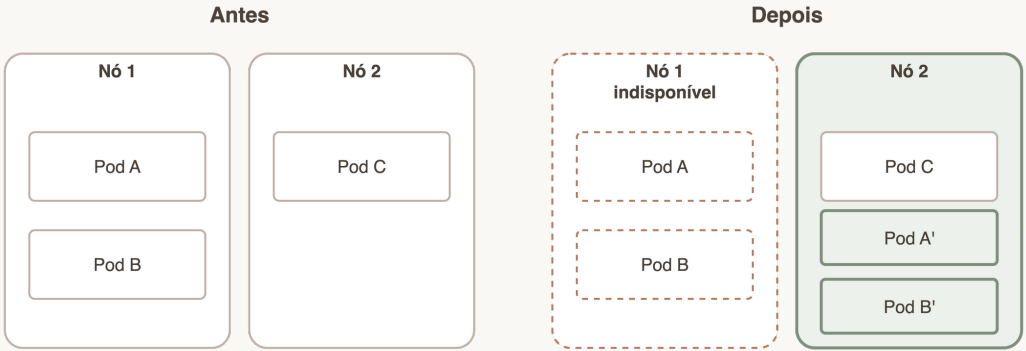
- O Kubernetes funciona por **reconciliação contínua**: compara permanentemente o *estado desejado* com o *estado observado*.
- Cada controlador observa o seu tipo de recurso e atua para reduzir a diferença entre os dois estados.
- O estado desejado fica registado no **etcd**, através do API server; o estado observado é reportado pelos kubelets.
- Consequência prática: apagar um Pod gerido por um ReplicaSet não o elimina de forma duradoura – o controlador recria-o.
- Este modelo, e não uma sequência de comandos, é o que dá ao Kubernetes a capacidade de self-healing.

O Ciclo, Visualmente



É esta repetição contínua — e não uma sequência de comandos — que dá self-healing.

Self-Healing na Prática: Perda de um Nó



Tipos de Service

- **ClusterIP** (por omissão): expõe o serviço apenas dentro do cluster, num IP virtual interno.
- **NodePort**: abre a mesma porta em todos os nós do cluster, tornando o serviço acessível a partir do exterior.
- **LoadBalancer**: pede um balanceador externo ao fornecedor de cloud; em clusters on-premises requer uma solução adicional.
- **Ingress**: não é um tipo de Service, mas um recurso próprio que encaminha tráfego HTTP/HTTPS por nome de host ou caminho.
- A seleção dos Pods de destino é feita por **labels e selectors**, não por endereços fixos.

Tipos de Service, Visualmente

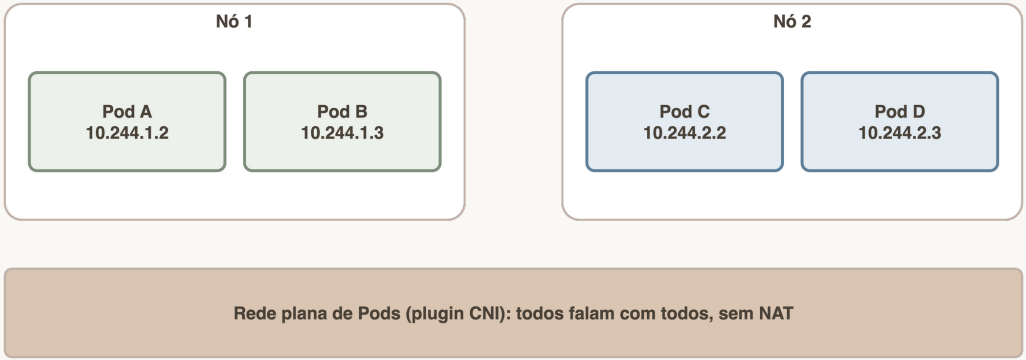


Ingress: um único ponto de entrada HTTP e HTTPS que encaminha por nome ou caminho para vários Services

O Modelo de Rede do Kubernetes

- Regra fundamental: cada Pod tem um endereço IP próprio e todos os Pods comunicam entre si sem NAT.
- Esta rede plana é implementada por um **plugin CNI** (Container Network Interface), não pelo Kubernetes em si.
- Exemplos de plugins CNI: Flannel, Calico, Weave Net, Cilium.
- **kube-proxy**: implementa em cada nó as regras que fazem um Service redirecionar tráfego para os Pods corretos.
- **CoreDNS**: fornece resolução de nomes interna, permitindo alcançar um Service pelo nome em vez do IP.
- Este modelo de rede será retomado na Aula 06, no contexto mais geral da SDN.

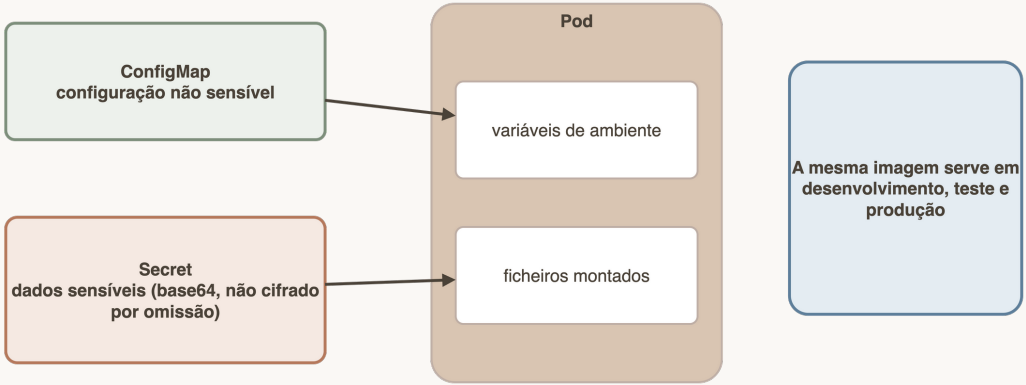
Rede Plana de Pods



Configuração e Segredos

- **ConfigMap**: guarda dados de configuração não sensíveis, separando-os da imagem do container.
- **Secret**: destinado a dados sensíveis (passwords, chaves, tokens).
- Atenção: por omissão, os Secrets são apenas codificados em base64 – *não* cifrados. É preciso ativar cifra em repouso no etcd e restringir acessos por RBAC.
- Ambos podem ser expostos ao container como variáveis de ambiente ou como ficheiros montados.
- Separar configuração da imagem permite usar a mesma imagem em desenvolvimento, teste e produção.

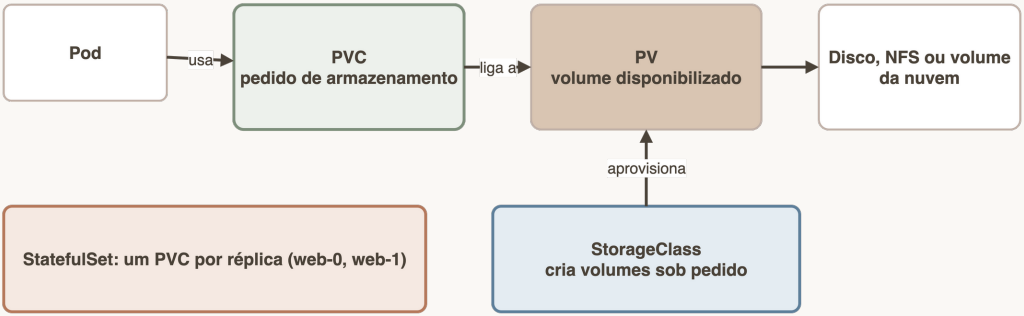
Configuração Fora da Imagem



Armazenamento Persistente

- Pods são efémeros: quando um Pod desaparece, o seu sistema de ficheiros local desaparece com ele.
- **PersistentVolume (PV)**: recurso de armazenamento disponibilizado no cluster.
- **PersistentVolumeClaim (PVC)**: pedido de armazenamento feito por uma aplicação, com capacidade e modo de acesso.
- **StorageClass**: permite aprovisionamento dinâmico, criando o volume automaticamente quando surge um PVC.
- **StatefulSet**: alternativa ao Deployment para aplicações com estado, que garante identidade e armazenamento estáveis por réplica.
- Estes conceitos ligam-se diretamente às Aulas 10 e 11, sobre armazenamento.

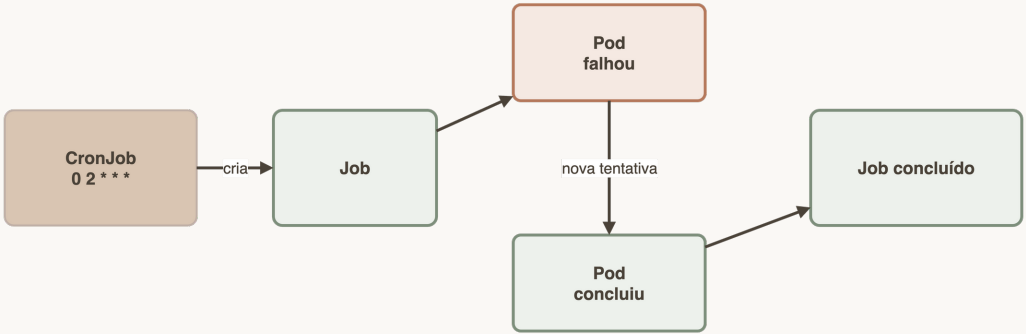
Do Pod ao Armazenamento



Cargas de Trabalho Não Contínuas: Job e CronJob

- Nem toda a carga de trabalho é um serviço permanente: um **Job** corre um Pod até este terminar com sucesso, em vez de o manter continuamente ativo.
- O Kubernetes garante que o Job é concluído mesmo que um Pod falhe a meio – recria-o e tenta de novo, respeitando um número máximo de tentativas.
- Útil para tarefas pontuais: migrações de base de dados, processamento em lote, geração de relatórios.
- **CronJob**: agenda Jobs para correr periodicamente, com uma sintaxe semelhante ao `cron` do Unix (ex.: backups noturnos).

Job e CronJob, Visualmente

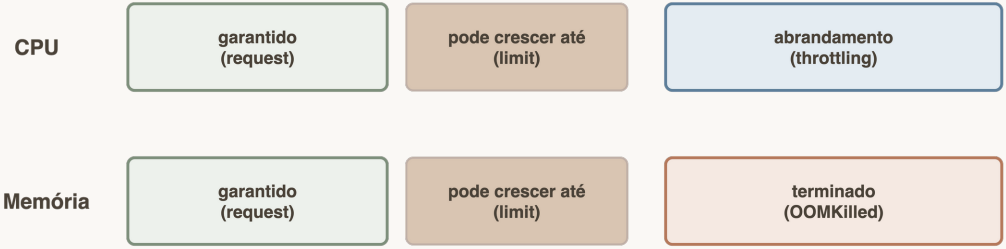


O Job repete até haver sucesso ou até ao limite de tentativas

Agendamento: Requests e Limits

- **Requests**: recursos que o Pod garantidamente precisa; o scheduler usa este valor para escolher um nó com capacidade suficiente.
- **Limits**: teto máximo que o container pode consumir, aplicado através dos cgroups vistos na Aula 04.
- Exceder o limite de memória leva à terminação do container (OOM kill); exceder o de CPU resulta apenas em *throttling*.
- **nodeSelector** e **affinity**: influenciam em que nós um Pod pode ou deve ser colocado.
- **Taints e tolerations**: mecanismo inverso, em que um nó repele Pods que não tolerem explicitamente a sua marca.

Requests e Limits, Visualmente



O request guia o scheduler. O limit é aplicado pelos cgroups.

Alternativas de Orquestração

- **Docker Swarm**: solução de orquestração nativa do Docker, mais simples de configurar, mas com ecossistema e funcionalidades mais limitadas.
- **HashiCorp Nomad**: orquestrador genérico, capaz de agendar não só containers mas também outras cargas de trabalho (binários, VMs).
- Kubernetes é hoje o standard de facto, com o ecossistema mais vasto e maior adoção na indústria.
- A escolha depende da complexidade do caso de uso e da curva de aprendizagem que a equipa está disposta a assumir.

Orquestradores em Comparação

Docker Swarm

nativo do Docker
simples de ativar
ecossistema reduzido

Nomad

executável único
containers e outras cargas
integra ferramentas HashiCorp

Kubernetes

padrão de facto
ecossistema vasto
mais complexo de operar



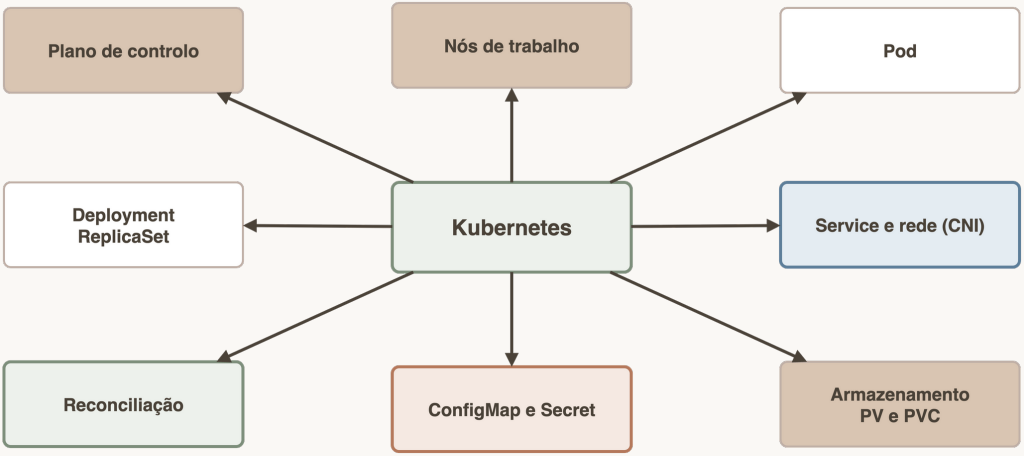
Síntese da Aula (1/2)

- Orquestração é necessária para gerir containers de forma fiável em múltiplos hosts.
- Kubernetes separa control plane (API server, etcd, scheduler, controller manager) de worker nodes (kubelet, kube-proxy, runtime).
- Pod, Deployment, ReplicaSet, Service e Namespace são os objetos centrais para descrever e expor aplicações.

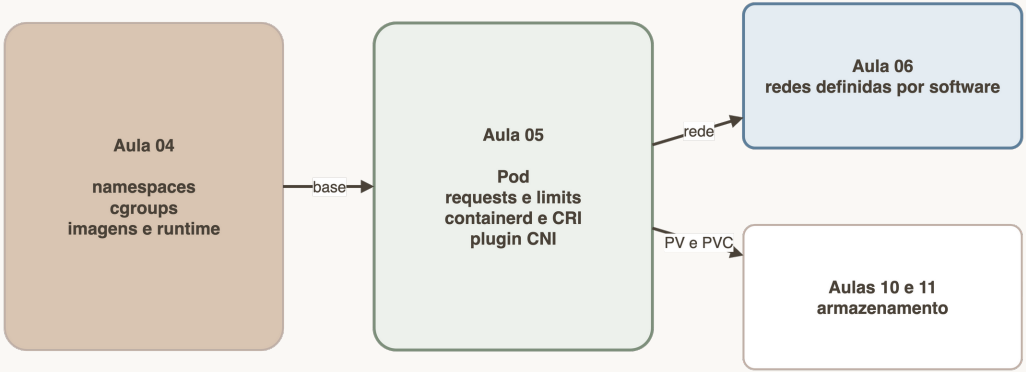
Síntese da Aula (2/2)

- O ciclo de reconciliação contínua entre estado desejado e observado é o que dá ao Kubernetes a capacidade de self-healing.
- Services (ClusterIP, NodePort, LoadBalancer) e o modelo de rede plana via CNI expõem e interligam Pods efémeros.
- ConfigMaps, Secrets e PersistentVolumes separam configuração e dados do ciclo de vida dos Pods; Jobs e CronJobs cobrem cargas de trabalho pontuais ou periódicas.
- Requests e limits guiam o agendamento e aplicam-se através dos cgroups vistos na Aula 04.

Mapa da Aula


Plano de controlo
Nós de trabalho
Pod
Deployment
ReplicaSet
Kubernetes
Service e rede (CNI)
Reconciliação
ConfigMap e Secret
Armazenamento
PV e PVC

Ligações a Outras Aulas



Obrigada.

Questões e comentários